

Anime Recommendation System

Petros Triantafyllos

Overview

- 1 Data Exploration
- 2 Algorithm and Hyperparameters
- 3 Testing & Evaluation
- 4 Results
- 5 Proposed Extensions

Kaggle Anime Dataset:

- Originally obtained from Kaggle (e.g., *Anime Recommendations Database*).
- Contains metadata on thousands of anime (title, genre, Scored By count, etc.)
- Includes user ratings for each anime (explicit feedback).

Enhanced via MyAnimeList Jikan API

- Scored By
- Time Aired

Two Datasets

- **Anime Dataset:** Contains `anime_id`, `name`, `genre`, `type`, `episodes`, `rating`, `members`, `Aired`, and `Scored By` (number of reviews).
- **Ratings Dataset:** Contains `user_id`, `anime_id`, and `rating`. (Values of -1 for missing ratings were removed.)
- datasets merged on `anime_id`.

Dimensionality Reduction:

- Anime with more than 1000 reviews (based on `Scored By`).
- Users with at least 50 total reviews.
- Items rated at least 100 times.

Visualizing the Data

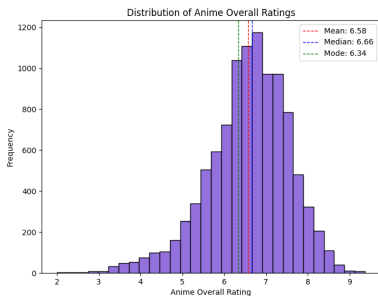


Figure: Distribution of Anime Overall Ratings

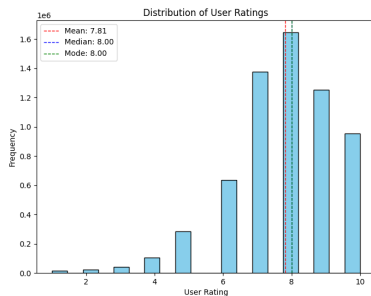


Figure: Distribution of User Ratings

Algorithm and Hyperparameters

Model:

- **Collaborative Filtering with SVD + Biases**
- *User Bias* b_u and *Item Bias* b_i estimated iteratively with regularization:

$$b_i \leftarrow \frac{\sum_u (r_{u,i} - \mu - b_u)}{\lambda + \text{count}}, \quad b_u \leftarrow \frac{\sum_i (r_{u,i} - \mu - b_i)}{\lambda + \text{count}}$$

- **Residual Matrix:** $R_{\text{res}} = R - \mu - b_u - b_i$
- **Truncated SVD:** Factorize the residual matrix into latent factors.

Hyperparameters:

- λ (Regularization), Number of latent factors: **Grid search, Bayes**
- Finally chosen: $\lambda = 20$, 100 latent factors.

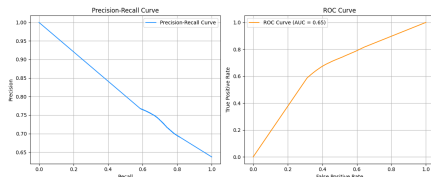
Testing and Evaluation

- **Leave-10-Out:** For each user, we randomly held out 10 ratings as test data, and used the rest for training.
- **Precision@10 / Recall@10:** On the test set, we recommend top-10 items.
 - $\text{Precision@10} = \frac{\# \text{ relevant in top-10}}{10}$
 - $\text{Recall@10} = \frac{\# \text{ relevant in top-10}}{\# \text{ total relevant}}$
- **ROC & PR Curves:** We introduced a threshold of relevancy to induce a binary relationship (e.g., rating ≥ 8 = relevant) and compute:
 - ROC AUC
 - Precision-Recall curve

Precision and Recall:

- Average Precision@5 ≈ 0.0215
- Average Recall@5 ≈ 0.0170

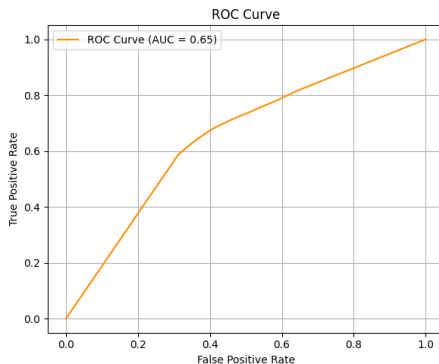
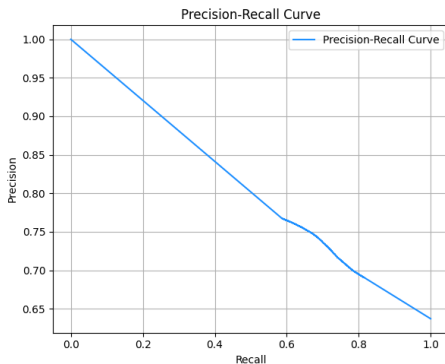
AUC: 0.65



Interpretation:

- *Precision-Recall Curve*: The trade-off between precision and recall as we vary the threshold.
- *ROC AUC = 0.65*: Above random guessing (0.5), but room for improvement.

PR Curves



Combining Cosine Similarity & Future Work

Item-based Cosine Similarity in Latent Space:

- After learning item latent vectors via SVD, compute $\text{cosine}(q_i, q_j)$ for items i and j .
- Use these similarities for *item-based CF*, e.g., recommending items most similar to those the user liked.
- Potentially combine SVD-based rating predictions with a similarity-based rerank or blend.

Future Work with Content and LLM Embeddings:

- Leverage genre, synopsis, and other metadata columns.
- Use Large Language Model (LLM) embeddings (e.g., from GPT) to encode textual descriptions of anime.
- Combine CF signals with content embeddings in a *hybrid model* to handle cold-start and improve overall accuracy.

Conclusion

- **Sparse Ratings:** Our dataset is large but ratings are concentrated in the 6–8 range.
- **Hybrid Approaches?:** Future work could incorporate anime metadata (genres, etc.) to address cold start and boost performance.
- **Parameter Tuning:** We found that $\lambda = 20$ and 100 latent factors gave a moderate AUC improvement (0.65).
- **Final Observations:**
 - Precision@10 $\approx 2\% + -0.02$
 - Recall@10 $\approx 1.7\% + -0.02$
 - ROC AUC $\approx 0.65 + -0.003$